COMPUTER ARCHITECTURE

VNU - UNIVERSITY *of* ENGINEERING *&* TECHNOLOGY

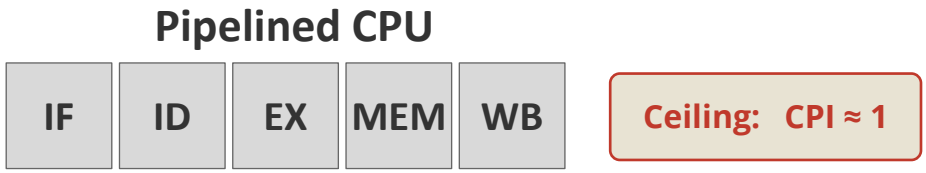
LECTURE 10: Parallelism

CONTENTS

- Quiz
- Instruction-level Parallelism (ILP)
  - Very Long Instruction Word (VLIW) processors
  - Superscalar processors
- Data-level Parallelism (DLP)
  - Single Instruction Multiple Data (SIMD) Instructions
- Thread-level Parallelism (TLP)
  - Data races
  - Locks

Quiz

Beyond Pipelining



> Pipelining: multiple instructions are executed concurrently (ILP)

- Theoretical speedup ≈ #stages
- Better **speedup**? *Superpipelining* (deeper pipeline, 10-15 stages)
  - Less work per stage: can shorten clock cycle (limited by power dissipation)
  - More hazard risks: more stalling – worsen CPI (> 1)

> Better CPI? **Multiple issue processors**

- Issue more than one instruction per cycle (i.e., **IPC > 1** or CPI < 1)
- Trade hardware complexity for lower CPI

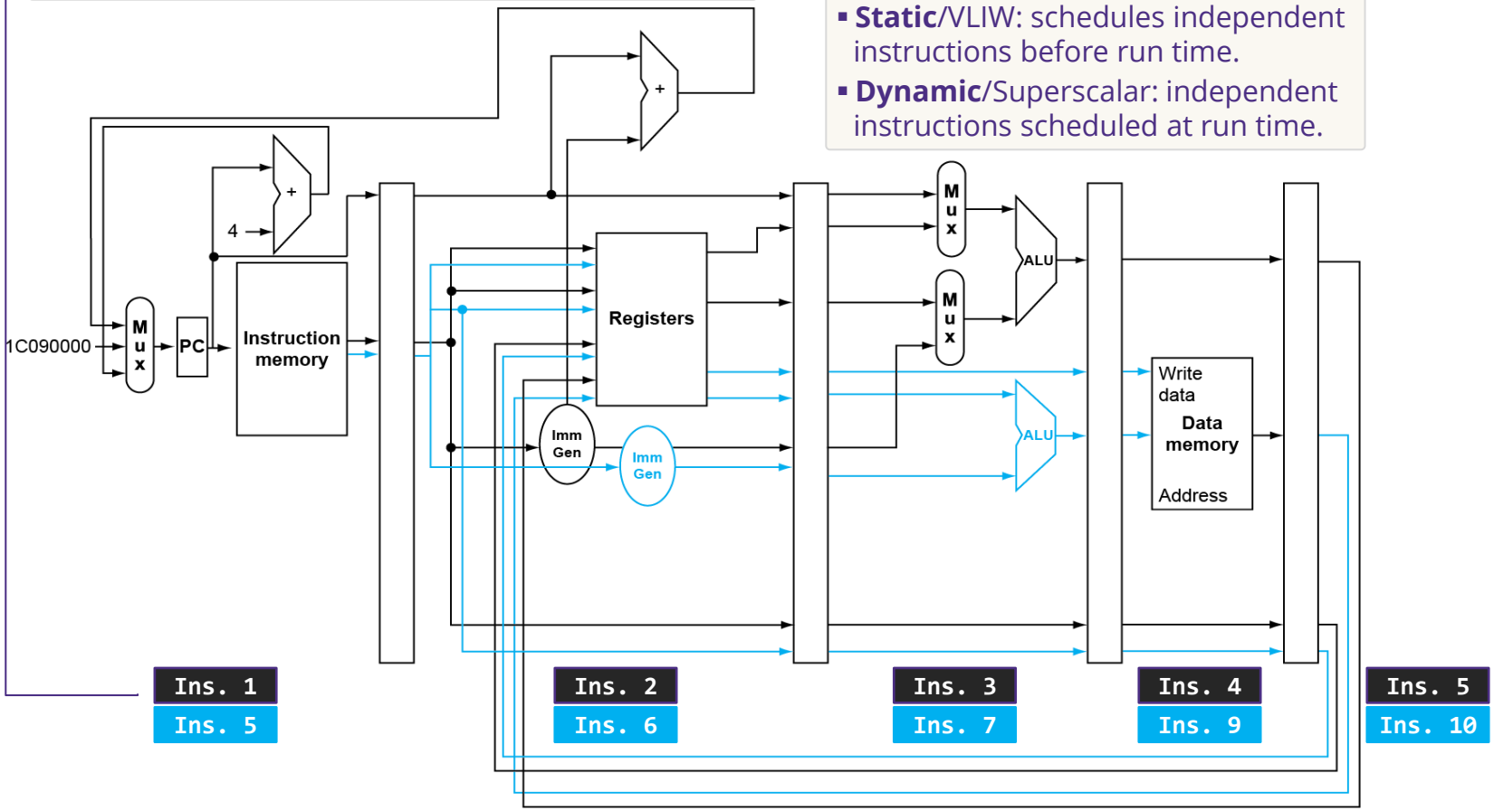
A Dual-Issue Datapath

Using 2 pipelines in parallel: IPC = $\frac{10 \text{ instructions}}{5 \text{ cycles}} = 2$

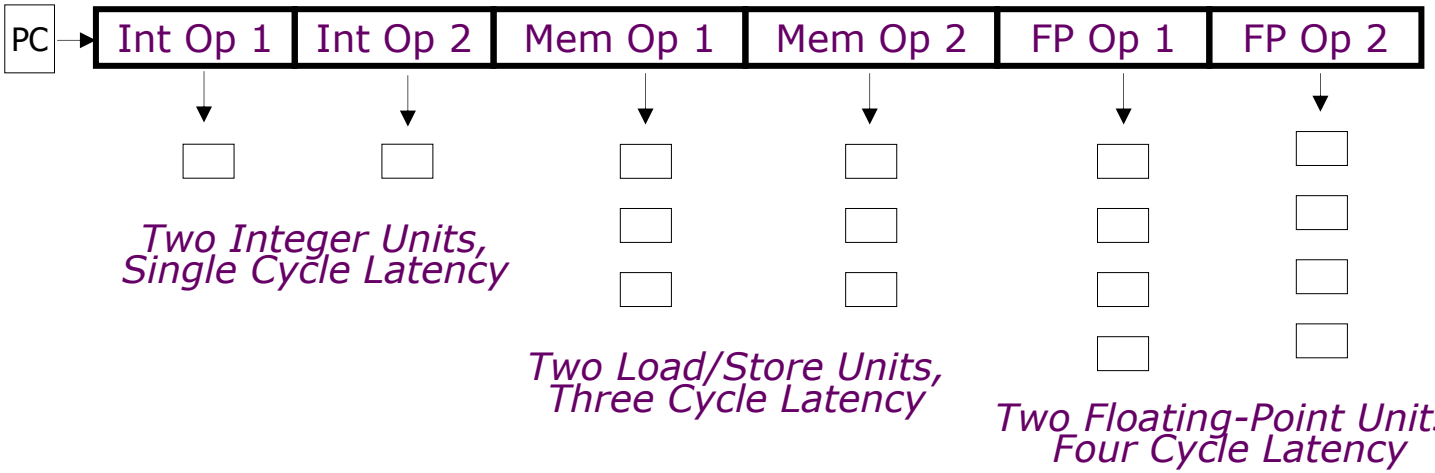
- **Dependencies** reduce peak IPC in practice

Two strategies to fight dependencies

- **Static**/VLIW: schedules independent instructions before run time.
- **Dynamic**/Superscalar: independent instructions scheduled at run time.



Very Long Instruction Word



> Compiler decides what runs in parallel before the program runs

- Compiler must remove some/all hazards & reorder instructions into issue packets
- Each slot is reserved for a specific operation type; empty slots are filled with nop.

> The CPU does not check for conflicts - it trusts the compiler.

- *Simpler hardware, smarter compiler.*

Compiler Scheduling for VLIW

```
Loop: lw   x31,0(x20)     // x31 = array element
      add  x31,x31,x21    // add scalar in x21
      sw   x31,0(x20)     // store result
      addi x20,x20,-4     // decrement pointer
      blt  x22,x20,Loop   // branch if x22 < x20
```

> **Example**: Dual-issue, including ALU/branch & Load/store slots

- Load-use hazard: still one cycle use latency, but now two instructions add & sw
- EX hazard: can't use ALU result and load/store in same packet → split add & sw

	ALU/branch	Load/store	cycle
Loop:	nop	lw x31,0(x20)	1
	add x31,x31,x21	nop	2
	addi x20,x20,-4	nop	3
	blt x22,x20,Loop	sw x31,4(x20)	4

❖ IPC = 5/4 = 1.25 (far from peak IPC = 2)

Loop Unrolling

> **Replicate** loop body to expose more independent work.

- Compiler must use different registers per replication (**register renaming**)
- Reduces loop-control overhead

	ALU/branch	Load/store	cycle
Loop:	addi x20,x20,-16	lw x28,0(x20)	1
	nop	lw x29,12(x20)	2
	add x28,x28,x21	lw x30,8(x20)	3
	add x29,x29,x21	lw x31,4(x20)	4
	add x30,x30,x21	sw x28,16(x20)	5
	add x31,x31,x21	sw x29,12(x20)	6
	nop	sw x30,8(x20)	7
	blt x22,x20,Loop	sw x31,4(x20)	8

❖ IPC = 14/8 = 1.75 (closer to peak IPC = 2)

❖ Cost: more registers, larger code size.

Dynamic Scheduling

> Disadvantages of static scheduling

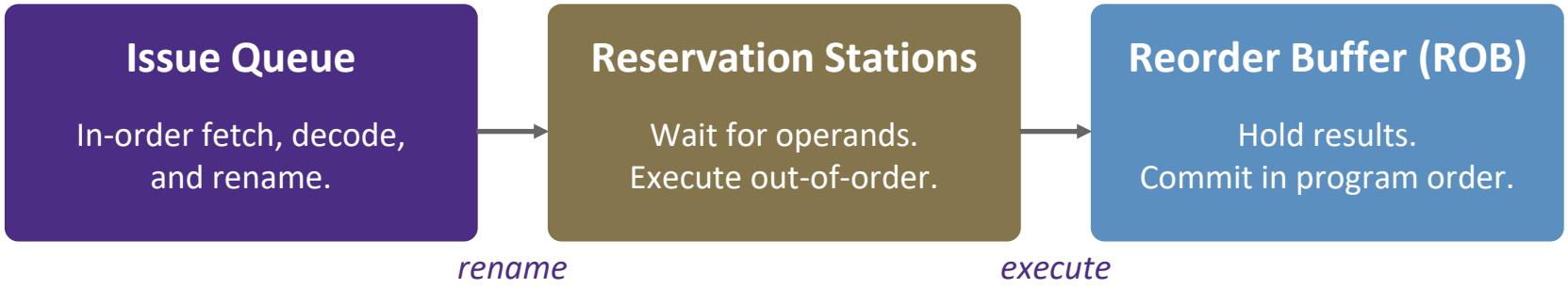
- Not all stalls are predicable
- Can’t always schedule around branches (outcome is dynamically determined)
- Different implementations of an ISA have different latencies and hazards

> Dynamic scheduling: Let hardware schedule

- The CPU finds independent instructions at run time
  - Fetch and decode multiple instructions per cycle.
  - Hardware decides who runs when.
- Out-of-order execution
  - Issue when operands are ready - not in program order.
  - Commit in program order → precise architectural state.
- Why this beats VLIW in practice
  - Cache misses and branches are not visible at compile time.
  - Same binary keeps working as hardware changes.

Superscalar: 3-Phase Pipeline

In-order Issue → Out-of-order Execute → In-order Commit



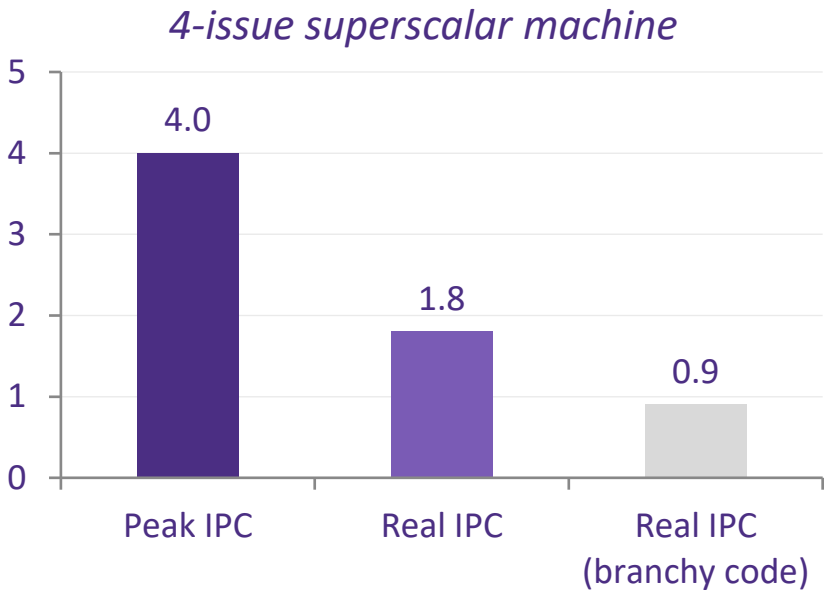
> **Issue Queue**: CPU fetches & decodes instructions *in program order*
  - *Rename* removes false (WAR/WAW) dependencies.

> **Reservation stations** hold instructions until operands arrive
  - Only cares about operand readiness → *out-of-order* execution.

> **ROB** guarantees precise state on exceptions, branches, interrupts
  - Keeps the program correct in the face of exceptions and bad guesses.

ILP Cost & Limits

Real-world IPC sits well below the peak the hardware advertises.



The ILP wall

- Branch mispredictions flush the pipeline.
- Cache misses stall hundreds of cycles.
- Wider issue → larger queues, more bypass paths.
- Complexity grows ~quadratically.
- Power and area do not scale with IPC.

The way forward

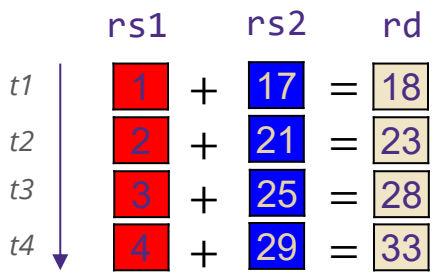
If instructions are hard to parallelize - *parallelize the data*.

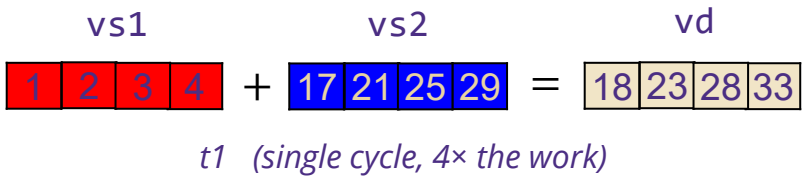
Data-Level Parallelism (DLP)

> Single-Instruction Multiple Data (SIMD)

- One instruction operates on many data elements at once.
- Vector registers hold N elements; a vector processor computes them in parallel
- Same control unit - much wider datapath
- Examples in practice: x86 AVX/AVX-512, ARM NEON/SVE, RISC-V RVV

> **RVV example**: `add rd, rs1, rs2` vs. `vadd.vv vd, vs2, vs1, vm`





- While parallel math provides a boost, the true speedup often comes from maximizing memory bandwidth through wide, contiguous loads and stores.

Code Vectorization

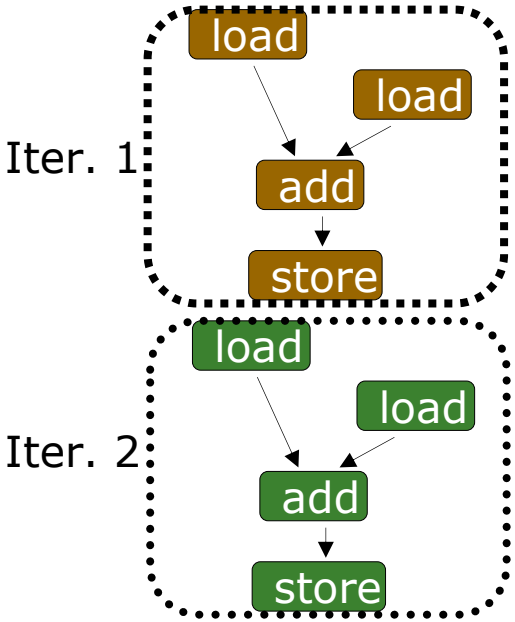
> SIMD instructions enable loop parallelization

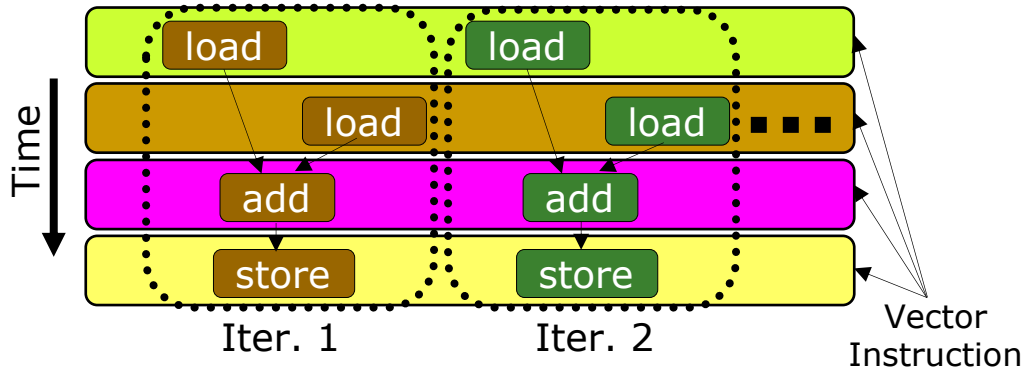
Scalar sequential code

```
for (i=0; i < N; i++) {
    C[i] = A[i] + B[i];
}
```

Vectorized code

```
for (i = 0; i < N; i += VLEN) {
    vC = vload(A+i) + vload(B+i);
    vstore(C+i, vC);}
```





DLP Cost & Limits

SIMD loves uniform work - it struggles when the work diverges.

Regular - SIMD shines

All 4 lanes do the same work → full utilization.

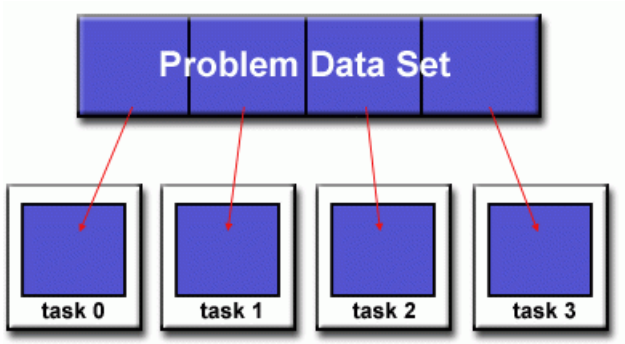
Divergent - SIMD stalls

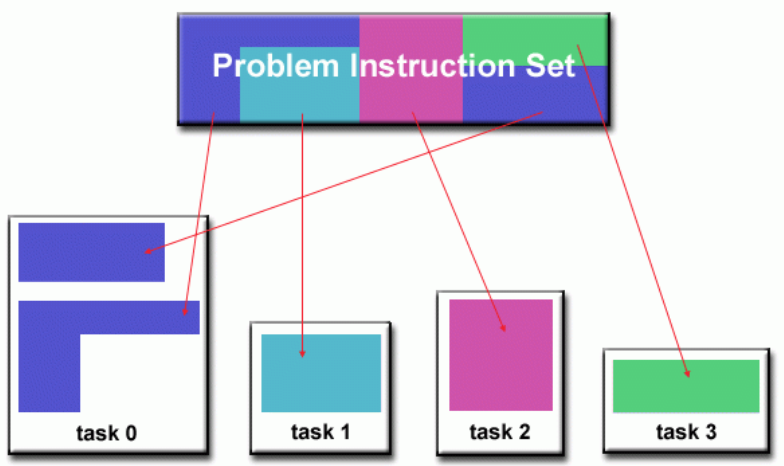
Branching → lanes masked off, hardware wasted.

> What SIMD cannot do well

- Conditional branches per element (control divergence).
- Irregular memory access (gather / scatter).
- Data dependencies between elements.
- Wide datapaths cost area even when underused.

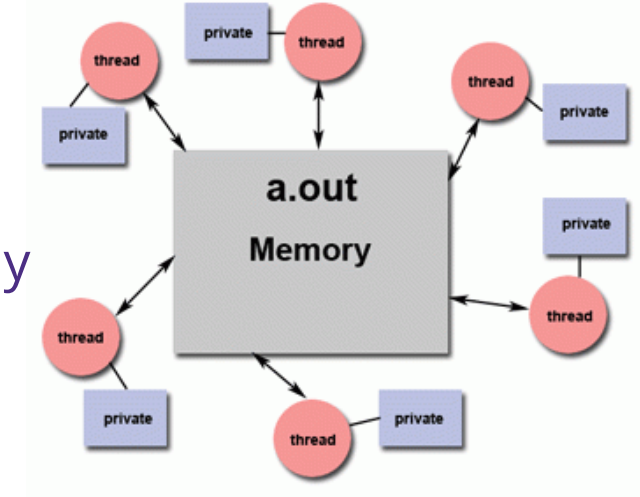
Thread-level Parallelism (TLP)





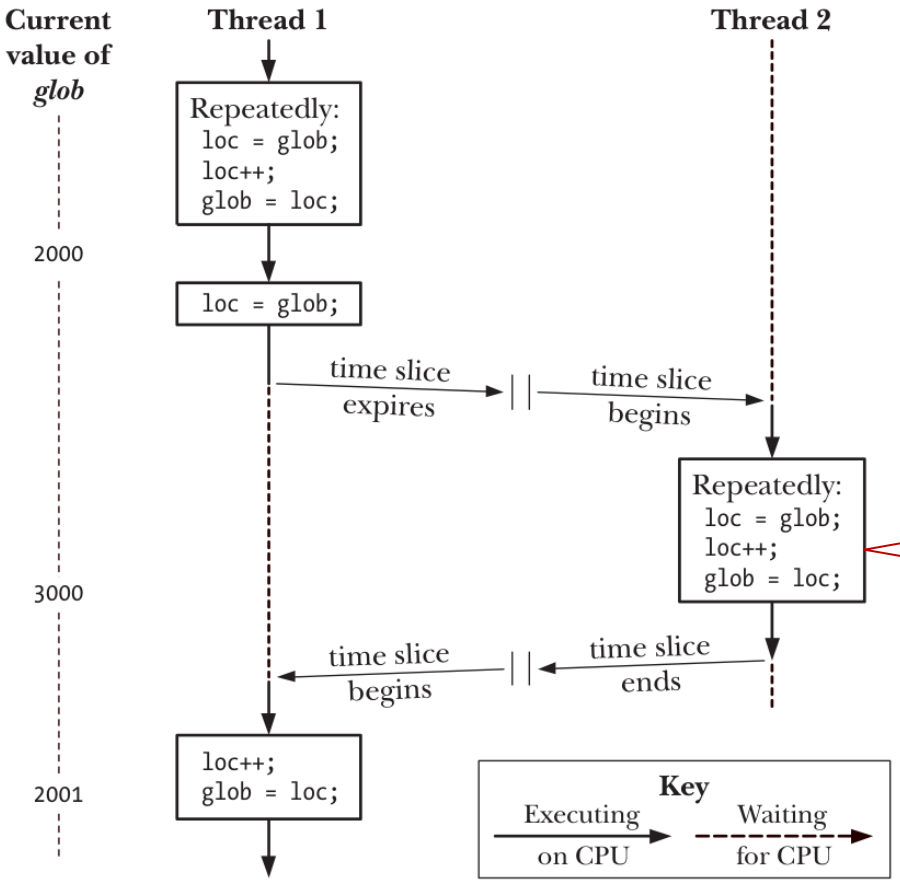
> Approaches to TLP:
  - Domain decomposition
  - Functional decomposition

> TLP runs multiple threads simultaneously
  - Each with its own PC but sharing memory.
  - The key hazard: data races



TLP: The Data Race

> What happens with this "simple" code, run on 2 threads?



- Two threads **race** to access and modify a shared variable (`glob`)
- The result depends on the *unpredictable* scheduling of threads

Race condition bugs are:
- *Rare*
- *Nondeterministic*
- *Silent-failing*

Why don't we change this to "`glob++`"?

Why the "Simple" Code Fails

```
for (j = 0; i < loops; j++)
    glob++;
```

- glob++ is **not** *atomic*
  - *Atomic operation*: executed as a single, indivisible step

Assembly code for thread i

```
    movq   (%rdi), %rcx
    testq  %rcx,%rcx
    jle    .L2
    movl   $0, %eax
-------------------------------
.L3:
    movq   cnt(%rip),%rdx
    addq   $1, %rdx
    movq   %rdx, cnt(%rip)
-------------------------------
    addq   $1, %rax
    cmpq   %rcx, %rax
    jne    .L3
.L2:
```

H_i : Head

L_i : ***Load*** glob
U_i : ***Update*** glob
S_i : ***Store*** glob

T_i : Tail

Race conditions can't be fixed by trick code

Resolving Data Races: Mutex Lock Idea

Buying milk analogy

- You & I share a fridge
- Always keep exactly *one* milk (never 0 or 2)

> Mutex lock: a synchronization primitive

- Used to protect (lock) a *critical section* of code
- Provides two operations: **acquire** and **release**
- Only ***one*** thread can hold the lock at any time; other threads are blocked until it is released.

> Acquire:

- If the lock is free, the thread takes it & access the critical section
- If another has it, the thread must wait until it's free.

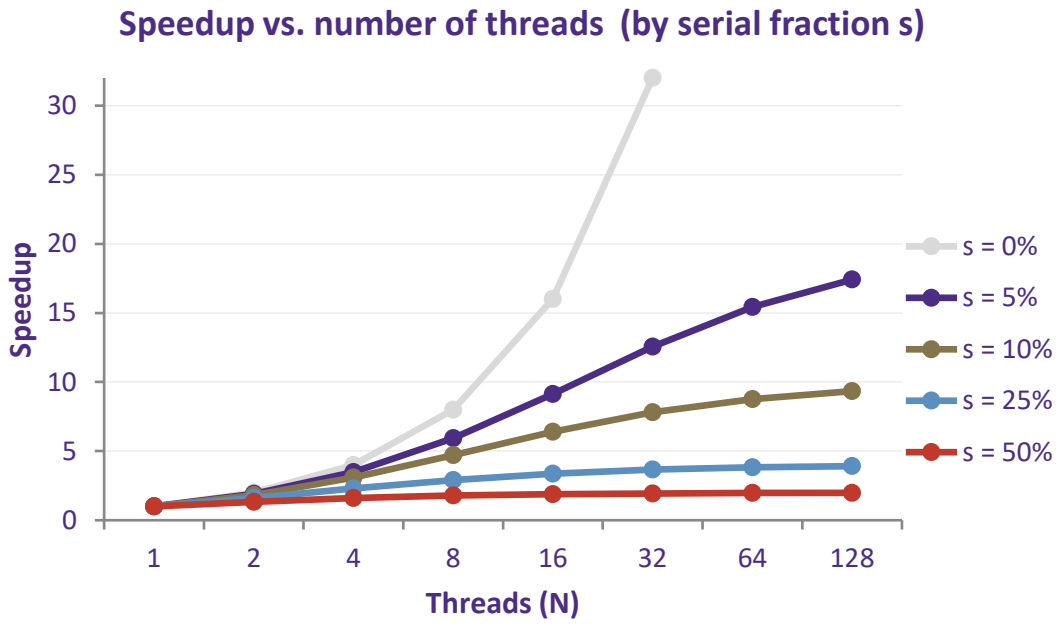
> Release:

- The thread exits the critical section
- The lock becomes available for other threads.

☜ Apply to “buying milk” example

TLP Cost & Limits

Adding cores helps - until the serial part dominates.



Amdahl's Law

Speedup ≤ 1 / (s + (1−s)/N)

The serial part is a hard ceiling.

Real-world costs

- Coherence traffic
- Lock contention
- Deadlock risk
- Load imbalance
- Cache-line ping-pong

Summary: The Three Levels of Parallelism

Each level fills a gap the previous one cannot fill.

Technique	What it gains	What it costs
ILP	More instructions per cycle (IPC > 1 from a single stream)	Dependency limits, hardware complexity, power, diminishing real-world IPC.
DLP	Same operation applied to many data elements in one instruction.	Requires regular data and uniform control. Wide datapaths cost area and power.
TLP	Multiple cores running multiple threads simultaneously.	Communication latency, synchronization, Amdahl's serial-fraction ceiling.

Cost rises as we go down the table. Modern systems combine all three.

Key takeaways

1. **Pipelining alone tops out at CPI ≈ 1.**
   Good ceiling - but not the end of the road.

2. **ILP pushes below CPI = 1.**
   Multiple instructions per cycle - limited by dependencies and complexity.

3. **DLP runs many data elements per op.**
   Same operation, wide datapath - needs regular data and uniform control.

4. **TLP uses multiple cores and threads.**
   Real work in parallel - bounded by communication and Amdahl's serial part.

5. **Modern performance combines all three.**
   ILP × DLP × TLP, plus locality, specialization, and energy efficiency.

NEXT LECTURE

[Flipped class] Memory hierarchy

- Pre-class
  - Homework
  - Self study
- In class
  - Reinforcement/enrichment discussion
  - Quizzes
- Post class
  - Homework
  - Consultation (if needed)